Adem Drpić

Razvoj softvera 2

02.03.2026.

Detalji implementacije sistema preporuke

Korisnik koji se tek registrovao na aplikaciju i nema historiju rezervacija inicijalno će dobiti **popularne preporuke**, odnosno usluge iz najbolje ocijenjenih salona koji su popularni na platformi. Prave personalizirane preporuke će se prikazati tek nakon što korisnik zakaže svoj prvi termin. Na osnovu prethodnih termina i ostavljenih recenzija, kreira se jedinstveni korisnički profil (UserProfile). Ovaj profil sabira učestalost ključnih riječi iz naziva usluga, prosječnu cijenu koju korisnik plaća, prosječno trajanje tretmana, kao i preferirane gradove u kojima korisnik posjećuje salone.

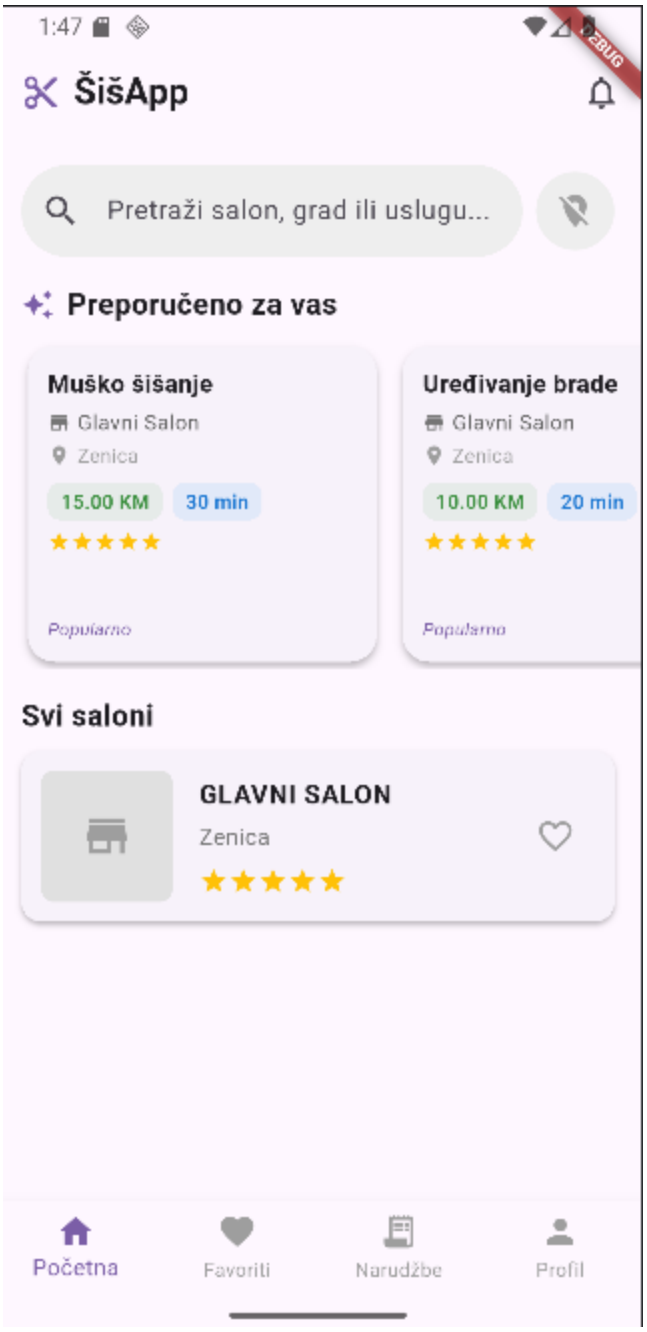
Zatim se računa sličnost sa svakom novom dostupnom uslugom koju korisnik prethodno nije iskoristio. Sistem boduje svaku uslugu (Content-Based Filtering) provjeravajući da li se ključne riječi poklapaju, da li je cijena i trajanje u rangu korisnikovih navika, te da li se salon nalazi u preferiranom gradu i da li ima visoku ocjenu (≥ 4.0). Na ovaj način korisniku se prikazuju visoko personalizirane preporuke koje tačno odgovaraju njegovom budžetu, vremenu i interesima. Sistem također uz svaku preporuku generiše i razlog (npr. *"Sličan cjenovni rang"*, *"Slične usluge koje ste koristili"*). Preporučene usluge se prikazuju na početnoj strani mobilne aplikacije.

Putanja do sistema preporuka: \RS2-main\backend\ŠišAppApi\Services\RecommendationService.cs

Putanja do dijela koda gdje se koristi sistem preporuke (API endpoint): \RS2-main\backend\ŠišAppApi\Controllers\RecommendationsController.cs

Mjesto u aplikaciji gdje se prikazuju rezultati sistema preporuke:

1. Ulogujte se na mobilnu aplikaciju sa kredencijalima:
    - Korisničko ime: mobile *(Korisnik sa historijom rezervacija)*
    - Lozinka: test
2. Na početnoj strani će se prikazati preporučene usluge i saloni u sekciji **"Preporučeno za vas"**.


1:47
DEBUG
ŠišApp
Pretraži salon, grad ili uslugu...
Preporučeno za vas
Muško šišanje
Glavni Salon
Zenica
15.00 KM
30 min
Popularno
Uređivanje brade
Glavni Salon
Zenica
10.00 KM
20 min
Popularno
Svi saloni
GLAVNI SALON
Zenica
Početna
Favoriti
Narudžbe
Profil

Print screen metoda GetReommendations I BuildUserProfile:

```
public async Task<List<RecommendationDto>> GetRecommendations(int userId, int maxResults = 10)
{
    var userAppointments = await _context.Appointments
        .Include(a => a.Service)
        .Include(a => a.Salon)
        .Where(a => a.UserId == userId && !a.IsDeleted && a.Status != "Cancelled")
        .ToListAsync();

    var userReviews = await _context.Reviews
        .Where(r => r.UserId == userId && !r.IsDeleted)
        .ToDictionaryAsync(r => r.AppointmentId, r => r.Rating);

    if (!userAppointments.Any())
    {
        return await GetPopularRecommendations(maxResults);
    }

    var profile = BuildUserProfile(userAppointments, userReviews);

    var usedServiceIds = userAppointments.Select(a => a.ServiceId).Distinct().ToHashSet();

    var candidateServices = await _context.Services
        .Include(s => s.Salon)
        .Where(s => !s.IsDeleted && s.IsActive
                    && s.Salon != null && s.Salon.IsActive && !s.Salon.IsDeleted
                    && !usedServiceIds.Contains(s.Id))
        .ToListAsync();

    var scored = new List<(Service service, float score, string reason)>();

    foreach (var service in candidateServices)
    {
        var (score, reason) = ScoreService(service, profile);
        if (score > 0)
        {
            scored.Add((service, score, reason));
        }
    }

    var topResults = scored
        .OrderByDescending(x => x.score)
        .Take(maxResults)
        .ToList();

    await PersistRecommendations(userId, topResults);

    return topResults.Select(x => new RecommendationDto
    {
        ServiceId = x.service.Id,
        ServiceName = x.service.Name,
        ServiceDescription = x.service.Description,
        Price = x.service.Price,
        DurationMinutes = x.service.DurationMinutes,
        SalonId = x.service.SalonId,
        SalonName = x.service.Salon!.Name,
        SalonCity = x.service.Salon.City,
        SalonRating = x.service.Salon.Rating,
        SalonImageIds = x.service.Salon.ImageIds,
        Reason = x.reason,
        RelevanceScore = x.score
    }).ToList();
}


private UserProfile BuildUserProfile(List<Appointment> appointments, Dictionary<int, int> reviews)
{
    var profile = new UserProfile();

    foreach (var apt in appointments)
    {
        if (apt.Service == null) continue;

        var words = apt.Service.Name.ToLower().Split(' ', StringSplitOptions.RemoveEmptyEntries);
        foreach (var w in words)
            profile.ServiceKeywords.TryAdd(w, 0);
        foreach (var w in words)
            profile.ServiceKeywords[w]++;

        profile.TotalPrice += apt.Service.Price;
        profile.PriceCount++;

        profile.TotalDuration += apt.Service.DurationMinutes;
        profile.DurationCount++;

        if (apt.Salon != null)
        {
            var city = apt.Salon.City.ToLower();
            profile.CityFrequency.TryAdd(city, 0);
            profile.CityFrequency[city]++;
        }

        profile.SalonVisits.TryAdd(apt.SalonId, 0);
        profile.SalonVisits[apt.SalonId]++;

        if (reviews.TryGetValue(apt.Id, out var rating))
        {
            profile.AvgRating += rating;
            profile.RatingCount++;
        }
    }

    return profile;
}
```

Print screen metoda ScoreService:

```
private (float score, string reason) ScoreService(Service service, UserProfile profile)
{
    float score = 0f;
    var reasons = new List<string>();

    var serviceWords = service.Name.ToLower().Split(' ', StringSplitOptions.RemoveEmptyEntries);
    int keywordHits = 0;
    foreach (var word in serviceWords)
    {
        if (profile.ServiceKeywords.TryGetValue(word, out var freq))
        {
            keywordHits += freq;
        }
    }
    if (keywordHits > 0)
    {
        score += Math.Min(keywordHits * 10f, 40f);
        reasons.Add("Slične usluge koje ste koristili");
    }

    if (service.Salon != null && service.Salon.Rating >= 4.0)
    {
        score += (float)(service.Salon.Rating * 4);
        reasons.Add($"Visoko ocijenjen salon ({service.Salon.Rating:F1}★)");
    }

    if (service.Salon != null)
    {
        var salonCity = service.Salon.City.ToLower();
        if (profile.CityFrequency.TryGetValue(salonCity, out var cityFreq))
        {
            score += Math.Min(cityFreq * 5f, 20f);
            reasons.Add($"U vašem gradu ({service.Salon.City})");
        }
    }

    if (profile.PriceCount > 0)
    {
        var avgPrice = profile.TotalPrice / profile.PriceCount;
        var priceDiff = Math.Abs(service.Price - avgPrice);
        if (priceDiff <= avgPrice * 0.3m)
        {
            score += 10f;
            reasons.Add("Sličan cjenovni rang");
        }
        else if (priceDiff <= avgPrice * 0.6m)
        {
            score += 5f;
        }
    }


    if (profile.DurationCount > 0)
    {
        var avgDuration = profile.TotalDuration / profile.DurationCount;
        var durationDiff = Math.Abs(service.DurationMinutes - avgDuration);
        if (durationDiff <= 15)
        {
            score += 10f;
            reasons.Add("Slično trajanje tretmana");
        }
        else if (durationDiff <= 30)
        {
            score += 5f;
        }
    }

    var reason = reasons.Any() ? string.Join(", ", reasons.Take(2)) : "Preporučeno";
    return (score, reason);
}
```

Metoda **BuildUserProfile** služi za kreiranje korisničkog profila analizirajući prošle termine i ostavljene recenzije. Izdvaja ključne riječi iz naziva usluga, sabira ukupan potrošeni novac i vrijeme, te mapira gradove i salone koje korisnik najčešće posjećuje. Ovaj profil služi kao baza/šablon za usporedbu sa drugim uslugama.

Metoda **ScoreService** je srž algoritma preporuke. Ona uspoređuje atribute svake dostupne usluge sa kreiranim korisničkim profilom. Dodjeljuje bodove (score) na osnovu preklapanja ključnih riječi, a zatim dodaje bodove ako je cijena u rasponu prosječne cijene profila, ako je trajanje slično, te ako se salon nalazi u najposjećenijem gradu korisnika. Uz bodovanje, metoda generiše i tekstualni razlog zašto je baš ta usluga preporučena.

Metoda **GetRecommendations** upravlja cjelokupnim procesom. Prvo provjerava da li korisnik ima historiju termina. Ako nema, poziva **GetPopularRecommendations**. Ako ima, generiše **UserProfile** i propušta sve nekorištene usluge kroz **ScoreService**. Nakon bodovanja, sortira usluge od najbolje prema najgoroj, uzima traženi broj najboljih (Top N) preporuka i pomoću metode **PersistRecommendations** ih spašava u bazu podataka brišući prethodne stare preporuke, osiguravajući da korisnik uvijek ima svježe podatke prilikom ulaska u aplikaciju.